

Taller de preparación para la Solemne 3

Ejercicios de Pandas, algoritmos, `np.roots` y `np.polyfit`

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 15, Sesión 2 (Sesión 30)

Organización del Taller

Ejercicios con Pandas

Ejercicios de Algoritmos

Ejercicios con `roots` y `polyfit`

Cierre

Organización del Taller

Organización del Taller

Contenidos del repaso

Pandas

- Inspección
- Filtros
- Datos faltantes
- **groupby**
- Gráficos

Algoritmos

- Búsqueda
- Recursividad
- Bubble Sort
- Merge Sort
- Complejidad

NumPy

- **np.roots**
- **np.polyfit**
- Residuos
- R^2
- Interpretación

Ejercicios con Pandas

Ejercicios con Pandas

Flujo de trabajo con un DataFrame

Cinco etapas

cargar → inspeccionar → limpiar → resumir → visualizar

- `head()`, `shape`, `columns`, `dtypes`
- `isna()`, `dropna()`, `fillna()`
- filtros con condiciones booleanas
- `groupby()` y `agg()`
- histogramas, scatter y boxplots

Ejercicios con Pandas

Datos para los ejercicios de Pandas

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 datos = {
6     "nombre": ["Sirio", "Canopus", "Arturo", "Vega",
7               "Capella", "Rigel", "Procyon", "Achernar"],
8     "clase": ["A", "F", "K", "A", "G", "B", "F", "B"],
9     "temperatura": [9940, 7350, 4290, 9600,
10                   4970, 12100, 6530, np.nan],
11     "magnitud": [-1.46, -0.74, -0.05, 0.03,
12                 0.08, 0.13, 0.34, 0.46],
13     "distancia_pc": [2.64, 95.0, 11.3, 7.68,
14                     13.0, 264.0, 3.51, 42.8]
15 }
16
17 estrellas = pd.DataFrame(datos)
```

Todos los ejercicios de esta sección usan el DataFrame `estrellas`.



Tareas

1. Muestre las primeras cinco filas.
2. Imprima las dimensiones y los tipos de datos.
3. Cuente los valores faltantes por columna.
4. Cree `estrellas_limpias` eliminando las filas sin temperatura.
5. Compruebe cuántas filas quedaron.

Pregunta de interpretación

¿Por qué conviene inspeccionar y limpiar los datos antes de calcular estadísticas?



Usando `estrellas_limpias`:

1. Seleccione las estrellas ubicadas a menos de 15 pc.
2. Seleccione las estrellas con temperatura mayor a 6000 K y distancia menor a 20 pc.
3. Cree una columna **brillante**:

True si `magnitud < 0`, False en otro caso.

4. Muestre `nombre`, `temperatura`, `distancia_pc` y `brillante`.

Cuidado

En filtros múltiples use paréntesis alrededor de cada condición y `&` en lugar de `and`.



Usando `estrellas_limpias`:

1. Agrupe por `clase`.
2. Calcule temperatura media, magnitud media y cantidad de estrellas.
3. Ordene el resumen por temperatura media de mayor a menor.
4. Construya un scatter de temperatura versus magnitud.
5. Use el color para distinguir la clase espectral.

Interpretación

Indique qué clase presenta la mayor temperatura media en este pequeño conjunto de datos.

Lista de verificación

- ¿Inspeccionó el DataFrame antes de modificarlo?
- ¿Creó una copia limpia sin perder el DataFrame original?
- ¿Usó paréntesis en los filtros múltiples?
- ¿Agrupó solamente después de limpiar los datos?
- ¿Incluyó etiquetas y leyenda en el gráfico?
- ¿Escribió una conclusión basada en los resultados?



```
1 print(estrellas.head())
2 print("Dimensiones:", estrellas.shape)
3 print(estrellas.dtypes)
4
5 print("Valores faltantes:")
6 print(estrellas.isna().sum())
7
8 estrellas_limpias = estrellas.dropna(
9     subset=["temperatura"]
10 ).copy()
11
12 print("Filas originales:", len(estrellas))
13 print("Filas limpias:", len(estrellas_limpias))
```

Interpretación

La inspección permite detectar columnas, tipos y datos faltantes antes de aplicar operaciones que podrían producir resultados incorrectos.



```
1 cercanas = estrellas_limpias[
2     estrellas_limpias["distancia_pc"] < 15
3 ]
4
5 calientes_cercanas = estrellas_limpias[
6     (estrellas_limpias["temperatura"] > 6000) &
7     (estrellas_limpias["distancia_pc"] < 20)
8 ]
9
10 estrellas_limpias["brillante"] = (
11     estrellas_limpias["magnitud"] < 0
12 )
13
14 columnas = ["nombre", "temperatura",
15             "distancia_pc", "brillante"]
16 print(estrellas_limpias[columnas])
```

Ejercicios con Pandas

Solución 3:

Resumen y visualización



```
1 resumen = estrellas_limpias.groupby(
2     "clase"
3 ).agg(
4     temperatura_media=(
5         "temperatura", "mean"
6     ),
7     magnitud_media=(
8         "magnitud", "mean"
9     ),
10    cantidad=( "nombre", "count" )
11 )
12
13 resumen = resumen.sort_values(
14     "temperatura_media",
15     ascending=False
16 )
17 print(resumen)
```

```
1 for clase, grupo in \
2     estrellas_limpias.groupby("clase"):
3     plt.scatter(
4         grupo["temperatura"],
5         grupo["magnitud"],
6         label=clase
7     )
8
9 plt.xlabel("Temperatura (K)")
10 plt.ylabel("Magnitud aparente")
11 plt.legend(title="Clase")
12 plt.grid(True, alpha=0.3)
13 plt.show()
```

En estos datos, la clase B presenta la mayor temperatura media.

Ejercicios de Algoritmos

Búsqueda

- Lineal: revisa uno por uno.
- Binaria: divide una lista ordenada.

Recursividad

- Caso base.
- Llamada con un problema menor.

Ordenamiento

- Bubble Sort: $\mathcal{O}(n^2)$.
- Merge Sort: $\mathcal{O}(n \log n)$.

Idea central

Dos programas correctos pueden necesitar cantidades de trabajo muy diferentes.



Considere la función:

```
suma_recursiva([4, 7, 2]).
```

1. Escriba una función recursiva que sume los elementos de una lista.
2. Identifique claramente el caso base.
3. Escriba las llamadas realizadas para la lista `[4, 7, 2]`.
4. Indique qué ocurriría si se elimina el caso base.

Pista

El problema más pequeño es sumar la lista sin su primer elemento.



Modifique el algoritmo para contar comparaciones e intercambios.

```
1 def bubble_sort(lista):
2     n = len(lista)
3     for i in range(n - 1):
4         for j in range(n - 1 - i):
5             if lista[j] > lista[j + 1]:
6                 lista[j], lista[j + 1] = \
7                     lista[j + 1], lista[j]
8     return lista
```

La nueva función debe retornar (`lista_ordenada`,
`comparaciones`, `intercambios`).



Use listas aleatorias con tamaños:

$$n = [10, 20, 40, 80, 160].$$

1. Ejecute **bubble_sort_instrumentado** para cada tamaño.
2. Guarde tamaños, comparaciones e intercambios usando **append()**.
3. Grafique las dos cantidades en función de n .
4. Compare el crecimiento observado con n^2 .
5. Explique por qué debe ordenar una copia de cada lista.

Lista de verificación

- ¿La función recursiva tiene un caso base alcanzable?
- ¿El contador de comparaciones está antes del `if`?
- ¿El contador de intercambios aumenta solo cuando hay intercambio?
- ¿La función retorna los tres resultados solicitados?
- ¿Usó `lista.copy()` para conservar los datos originales?
- ¿El gráfico permite reconocer la tendencia de crecimiento?



```
1 def suma_recursiva(lista):
2     # Caso base
3     if len(lista) == 0:
4         return 0
5
6     # Caso recursivo
7     return lista[0] + suma_recursiva(lista[1:])
8
9 print(suma_recursiva([4, 7, 2]))
```

Seguimiento

$$S([4, 7, 2]) = 4 + S([7, 2]) = 4 + 7 + S([2]) = 4 + 7 + 2 + S([]) = 13.$$

Sin caso base, las llamadas no terminan y Python genera un `RecursionError`.



```
1 def bubble_sort_instrumentado(lista):
2     n = len(lista)
3     comparaciones = 0
4     intercambios = 0
5
6     for i in range(n - 1):
7         for j in range(n - 1 - i):
8             comparaciones += 1
9
10            if lista[j] > lista[j + 1]:
11                lista[j], lista[j + 1] = \
12                    lista[j + 1], lista[j]
13                intercambios += 1
14
15     return lista, comparaciones, intercambios
```



```
1 np.random.seed(15)
2 tamanos = [10, 20, 40, 80, 160]
3 comparaciones = []
4 intercambios = []
5
6 for n in tamanos:
7     datos = np.random.randint(
8         0, 1000, size=n
9     ).tolist()
10
11     ordenada, comp, inter = \
12         bubble_sort_instrumentado(
13             datos.copy()
14         )
15
16     comparaciones.append(comp)
17     intercambios.append(inter)
```

```
1 plt.plot(
2     tamanos, comparaciones, "o-",
3     label="comparaciones"
4 )
5 plt.plot(
6     tamanos, intercambios, "s--",
7     label="intercambios"
8 )
9 plt.plot(
10    tamanos, np.array(tamanos)**2,
11    "k:", label="referencia n^2"
12 )
13 plt.xlabel("Tamaño n")
14 plt.ylabel("Número de operaciones")
15 plt.legend()
16 plt.grid(True, alpha=0.3)
17 plt.show()
```

Ejercicios con `roots` y `polyfit`

Ejercicios con `roots` y `polyfit`

¿Qué herramienta corresponde?

`np.roots`

- Tenemos una ecuación polinómica.
- Conocemos sus coeficientes.
- Buscamos dónde vale cero.

`np.polyfit`

- Tenemos pares de datos x, y .
- Elegimos el grado del modelo.
- Buscamos sus coeficientes.

Pregunta guía

¿Tengo una ecuación para resolver o mediciones para ajustar?

Ejercicio 7:

Altura de un objeto



La altura de un objeto está dada por

$$h(t) = -4.9t^2 + 16t + 2.$$

1. Escriba los coeficientes en el orden requerido por `np.roots`.
2. Calcule las raíces de $h(t) = 0$.
3. Conserve solamente las raíces reales.
4. Seleccione la solución físicamente válida.
5. Explique qué representa esa raíz.

Ejercicios con roots y polyfit

Datos para los ejercicios de ajuste

```
1 t = np.array([0, 1, 2, 3, 4, 5, 6])
2 x = np.array([1.2, 3.0, 4.8, 6.9,
3               8.7, 10.6, 12.5])
4
5 t2 = np.array([0, 1, 2, 3, 4, 5])
6 y2 = np.array([1.1, 3.4, 7.8, 14.1,
7                22.3, 32.2])
```

- `t`, `x`: movimiento aproximadamente uniforme.
- `t2`, `y2`: movimiento aproximadamente cuadrático.



Con los datos $\mathbf{t}$ y $\mathbf{x}$:

1. Ajuste $x(t) = vt + x_0$ usando `np.polyfit`.
2. Extraiga e interprete v y x_0 .
3. Calcule los valores ajustados.
4. Calcule residuos y R^2 .
5. Grafique datos y modelo en la misma figura.

Interpretación

¿Qué representa la pendiente en este contexto?



Con los datos `t2` y `y2`:

1. Ajuste $y(t) = at^2 + bt + c$ usando `deg=2`.
2. Calcule el ajuste sobre una malla fina.
3. Grafique datos y modelo.
4. Use los coeficientes ajustados para resolver $y(t) = 20$.
5. Conserve tiempos reales dentro del intervalo medido.

Pista

Para resolver $at^2 + bt + c = 20$, use los coeficientes $[a, b, c - 20]$.

Ejercicios con roots y polyfit

Antes de mirar la solución: ajuste

Lista de verificación

- ¿Escribió los coeficientes desde la potencia mayor?
- ¿Incluyó ceros para las potencias ausentes?
- ¿Diferenció raíces matemáticas de soluciones físicas?
- ¿Construyó las predicciones antes de calcular R^2 ?
- ¿Graficó datos y ajuste en la misma figura?
- ¿Interpretó los coeficientes con sus unidades?

Ejercicios con roots y polyfit

Solución 7:



Altura de un objeto

```
1 coef = [-4.9, 16, 2]
2 raices = np.roots(coef)
3
4 raices_reales = []
5 for raiz in raices:
6     if abs(raiz.imag) < 1e-10:
7         raices_reales.append(raiz.real)
8
9 tiempos_fisicos = []
10 for tiempo in raices_reales:
11     if tiempo >= 0:
12         tiempos_fisicos.append(tiempo)
13
14 print("Raíces:", raices)
15 print("Tiempo de impacto:", tiempos_fisicos[0])
```

La raíz positiva representa el instante posterior al lanzamiento en que el objeto llega al suelo.

Ejercicios con roots y polyfit

Solución 8:

Ajuste lineal y R^2



```
1 v, x0 = np.polyfit(t, x, deg=1)
2 x_fit = v*t + x0
3
4 residuos = x - x_fit
5 SS_res = np.sum(residuos**2)
6 SS_tot = np.sum(
7     (x - np.mean(x))**2
8 )
9 R2 = 1 - SS_res/SS_tot
10
11 print(f"v = {v:.3f}")
12 print(f"x0 = {x0:.3f}")
13 print(f"R2 = {R2:.4f}")
```

```
1 plt.scatter(
2     t, x, label="datos"
3 )
4 plt.plot(
5     t, x_fit, "r-",
6     label="ajuste lineal"
7 )
8 plt.xlabel("Tiempo")
9 plt.ylabel("Posición")
10 plt.legend()
11 plt.grid(True, alpha=0.3)
12 plt.show()
```

La pendiente v representa la velocidad estimada.

Ejercicios con roots y polyfit

Solución 9:

Ajuste cuadrático y umbral



```
1 a, b, c = np.polyfit(
2     t2, y2, deg=2
3 )
4
5 t_fino = np.linspace(0, 5, 200)
6 y_fino = (
7     a*t_fino**2 +
8     b*t_fino + c
9 )
10
11 coef_umbral = [a, b, c - 20]
12 raices = np.roots(coef_umbral)
13
14 tiempos = []
15 for raiz in raices:
16     if abs(raiz.imag) < 1e-10:
17         if 0 <= raiz.real <= 5:
18             tiempos.append(raiz.real)
```

```
1 plt.scatter(
2     t2, y2, label="datos"
3 )
4 plt.plot(
5     t_fino, y_fino, "r-",
6     label="ajuste grado 2"
7 )
8 plt.axhline(
9     20, color="black",
10    linestyle="--", label="y = 20"
11 )
12 plt.xlabel("Tiempo")
13 plt.ylabel("Posición")
14 plt.legend()
15 plt.grid(True, alpha=0.3)
16 plt.show()
17
18 print("Tiempos válidos:", tiempos)
```

Cierre

Datos y algoritmos

- No revisar valores faltantes.
- Usar **and** en filtros de Pandas.
- Omitir el caso base.
- Ubicar mal los contadores.
- Modificar la lista original.

Raíces y ajustes

- Desordenar coeficientes.
- Aceptar raíces no físicas.
- Confundir **roots** con **polyfit**.
- No calcular predicciones.
- Presentar un gráfico sin interpretación.

Puedo resolver una pregunta si...

- identifico los datos de entrada y la salida solicitada;
- selecciono la herramienta adecuada;
- separo la solución en pasos;
- ejecuto y verifico cada paso;
- incluyo etiquetas, unidades y leyendas;
- explico el significado del resultado.

Última revisión

Un programa no está completo solamente porque ejecuta: también debe responder la pregunta planteada.

- **Pandas:** inspeccionar, limpiar, filtrar, agrupar y visualizar.
- **Algoritmos:** recursividad, instrumentación y crecimiento del costo.
- **np.roots:** resolver ecuaciones e interpretar raíces.
- **np.polyfit:** ajustar datos y evaluar el modelo.

Habilidad integrada

Traducir un problema de física, astronomía o análisis de datos en una secuencia clara de operaciones computacionales.